

MA SOFT TECH SOLUTIONS

DEVOPS ENGINEERING

INTERNSHIP ASSESSMENT TASK

Practical Assessment — Build, Containerize, Automate, Troubleshoot & Design for Production

Assessment Details	
Assessment Type	Hands-on Technical Assessment
Level	Internship / Entry-Level DevOps
Assessment Duration	3 Hours
Submission	Complete repository + documentation + evidence
Evaluation	Technical review + live walkthrough

Candidate Name: _____

Date: _____ **Evaluator:** _____

Assessment Brief

Scenario: You have joined MA SOFT TECH SOLUTIONS as a DevOps Engineer. A development team needs a reliable system for building, testing, containerizing and deploying a web-based application. There is currently no DevOps infrastructure or deployment pipeline.

Your responsibility is to design and implement the complete workflow from source code to deployment, while ensuring that the system is testable, secure, recoverable and production-ready.

You are free to select the programming language, framework and tools. Use practical, explainable choices rather than unnecessary complexity.

General Expectations

- Work should be reproducible by another developer using your documentation.
- Use meaningful names, clean repository structure and clear commit history.
- Do not commit passwords, API keys, tokens, private keys or cloud credentials.
- Where a paid cloud resource is not available, provide a realistic deployment simulation and production architecture.
- Evidence should show both successful operation and the troubleshooting of at least one intentionally introduced failure.
- Be prepared to explain design decisions, commands, pipeline behavior and troubleshooting logic during the assessment.

PART 1 — SYSTEM DESIGN

Before implementation, design the complete solution. Your design must define:

- Application technology stack
- Project / repository structure
- Git workflow
- Testing strategy
- Docker / container strategy
- CI/CD pipeline
- Configuration and environment-variable strategy
- Health-check strategy
- Deployment strategy
- Rollback strategy
- Logging and monitoring strategy
- Production cloud architecture

Architecture Diagram

Create an architecture diagram showing:

Developer → Git Repository → CI/CD → Testing → Docker Build → Container Registry → Deployment → Health Check → Monitoring

Explain the purpose of every major component.

PART 2 — BUILD AN APPLICATION FROM SCRATCH

Create a small but functional web/API application. You may select Python, Node.js, Java, Go, PHP or another suitable technology.

Required Features

- Main application endpoint
- /health or equivalent health endpoint
- At least one API endpoint

- JSON response
- Input validation
- Appropriate HTTP status codes
- Environment-variable based configuration
- At least one automated test

The application should be organized professionally so another developer can understand and run it without assistance.

PART 3 — GIT & VERSION CONTROL

Create the Git repository completely from scratch.

Requirements

- Proper repository structure
- Meaningful commit messages
- .gitignore
- README.md
- Separate branch for at least one feature/fix
- Merge the feature/fix into the main development branch

Explain

- How you would protect the main branch in a professional software company
- Pull requests
- Code review
- Branch protection
- Merge conflicts
- Version / tag management

PART 4 — LINUX SYSTEM ADMINISTRATION

Use a Linux environment available to you.

Demonstrate

- Navigate the filesystem
- Create and manage files/directories
- Manage permissions
- Inspect running processes
- Identify network ports
- Manage environment variables
- Inspect application logs
- Identify resource/process problems

Incident Scenario

Assume the server is reachable, but users cannot access the application. Develop a logical troubleshooting process.

- Application failure
- Process failure
- Port / listening failure
- Configuration failure
- Permission problem
- Dependency problem
- Container / runtime problem

Document the commands or methods you would use and explain why you would use them in that order.

PART 5 — ADVANCED DOCKERIZATION

Containerize the application completely from scratch.

Create

- Dockerfile
- .dockerignore
- Required container configuration
- Correct service port exposure
- Environment-variable support
- Health-check mechanism
- Understandable logs
- Secure runtime configuration where practical

Docker Optimization Challenge

Implement at least three improvements related to:

- Image size
- Build speed
- Layer caching
- Security
- Dependency management
- Runtime permissions

Explain the reason behind each optimization.

PART 6 — CI/CD PIPELINE

Create a complete CI/CD pipeline from scratch using a free CI/CD platform/tool available to you, such as GitHub Actions.

Required Pipeline Flow

Git Push → Checkout Source Code → Install Dependencies → Run Automated Tests → Code/Configuration Validation → Build Docker Image → Validate Container → Health Check → Deployment / Deployment Simulation

The pipeline should automatically execute after a code push.

Evidence

- Successful execution
- Failed execution
- Error information
- Successful execution after fixing the problem

PART 7 — FAILURE INJECTION & TROUBLESHOOTING

After successfully completing the pipeline, intentionally introduce one realistic failure.

Possible Failures

- Wrong port
- Broken test
- Invalid environment variable
- Dependency conflict
- Docker startup error
- Application configuration error

- Health-check failure
- Permission problem
- CI/CD configuration problem

Required Lifecycle

Failure → Collect Evidence → Form Hypothesis → Investigate → Identify Root Cause → Apply Fix → Rebuild → Retest → Verify Successful Deployment

Explain why the failure occurred and show the evidence that led to the root-cause conclusion.

PART 8 — CLOUD PRODUCTION ARCHITECTURE

You do not need a paid cloud account. Design how the application would be deployed in a real production cloud environment.

Choose and Justify

- Cloud VM + Docker
- Managed Container Service
- Kubernetes
- Another suitable production architecture

Address

- Networking
- Application access
- Ports
- Internal / external communication
- Security
- Secrets
- Authentication
- Least privilege and access control
- Deployment
- Container image and registry
- CI/CD
- Reliability
- Health checks
- Failure detection
- Restart strategy
- Rollback
- Horizontal scaling
- Load balancing
- Resource management
- Application logs
- Error monitoring
- CPU / memory usage
- Health status

PART 9 — SECURITY CHALLENGE

#	Question
1	Why should passwords, API keys and cloud credentials never be committed to Git?
2	Where should CI/CD secrets be stored?

#	Question
3	What is the principle of least privilege?
4	What is the difference between authentication and authorization?
5	What should happen if a production deployment fails its health check?
6	How would you prevent a compromised CI/CD credential from accessing the entire infrastructure?
7	What security risks can exist inside Docker containers?

PART 10 — DEPLOYMENT STRATEGY

Design a safe production deployment strategy. Compare at least two of the following:

Strategy	Discuss
Rolling Deployment	When to use; advantages; disadvantages; rollback; downtime considerations
Blue-Green Deployment	When to use; advantages; disadvantages; rollback; downtime considerations
Canary Deployment	When to use; advantages; disadvantages; rollback; downtime considerations

Then select one strategy for your application and justify your decision.

PART 11 — PRODUCTION INCIDENT

Imagine that a new version has been deployed. Five minutes later:

- Application response time increases
- Error rate increases
- Some users cannot access the service

As the DevOps Engineer, explain:

- What would you check first?
- What logs/metrics would you inspect?
- How would you determine whether the problem is caused by the application or infrastructure?
- Would you rollback? How and why?
- How would you verify that the incident is resolved?
- What would you do after the incident to prevent recurrence?

FINAL DELIVERABLES

- Complete source-code repository
- README.md
- Git history
- Automated tests
- Dockerfile
- .dockerignore
- CI/CD workflow
- Architecture diagram
- Deployment design
- Evidence/screenshots of successful pipeline
- Evidence/screenshots of failed pipeline
- Failure troubleshooting explanation
- Cloud production architecture

- Security considerations
- Rollback/deployment strategy